

Project Inception: Solution Proposal

Topic	Community Chat Application
Prepared by	Ronald Thian
Prepared on	19 January 2026
Approved by	
Approved on	

Context

In January 2026, a community chat application project request was raised. The main purpose of this application is to empower leaders within the community and grouping different branches of people into their dedicated group, or community. Leaders are able to quickly disseminate information and monitor the flow of information, hence, delegating tasks expected of the System Administrator.

Problem Evaluation

Several existing solutions and chat frameworks were proposed by the business team. These solutions provide easy implementation of a chatroom-like system, aligned more towards a Wechat or Discord clone.

However, many of these solutions use a large suite of different services like no-sql databases, queues like kafka, and written in languages such as Golang. Also, unique features not commonly found in these chatroom applications are required, like the restriction of peer-to-peer chatting, management of the different message deletion times within these communities, and, blocking repeated rejected join requests.

*Update: After clarification with the Project Manager, the rough numbers that we are expecting to support is around **1000 users**.*

Proposed Solution

Solution 1 - OpenIM or other open sourced frameworks

Open-sourced chat applications are easy to spin up and use, they provide full fledged features like private chats, group chats and friend requests. However, most of the existing

applications are written in languages outside of our team’s capabilities, for example, Golang is used for OpenIM [1]. Teams that rely on these frameworks usually are expecting millions of concurrent users, hence, an elaborate infrastructure setup is preferred.

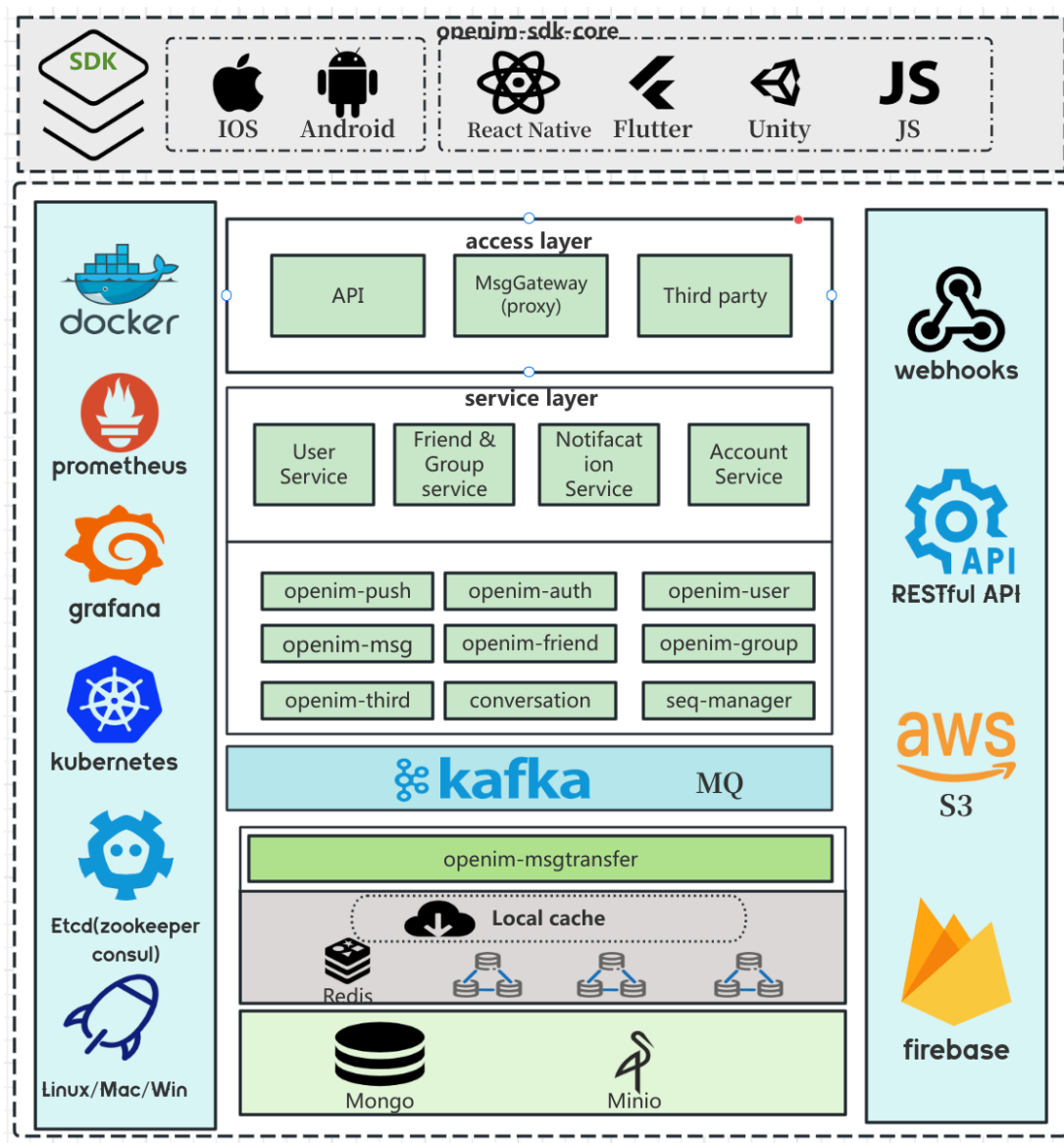


Figure 1. Overview of OpenIM chat application framework

Although these frameworks boast high throughput and stability, tweaking it to fit the unique requirements of this project risks incidents that the team cannot resolve as majority of it is written in Rust or Golang.

Solution 2a - NodeJS

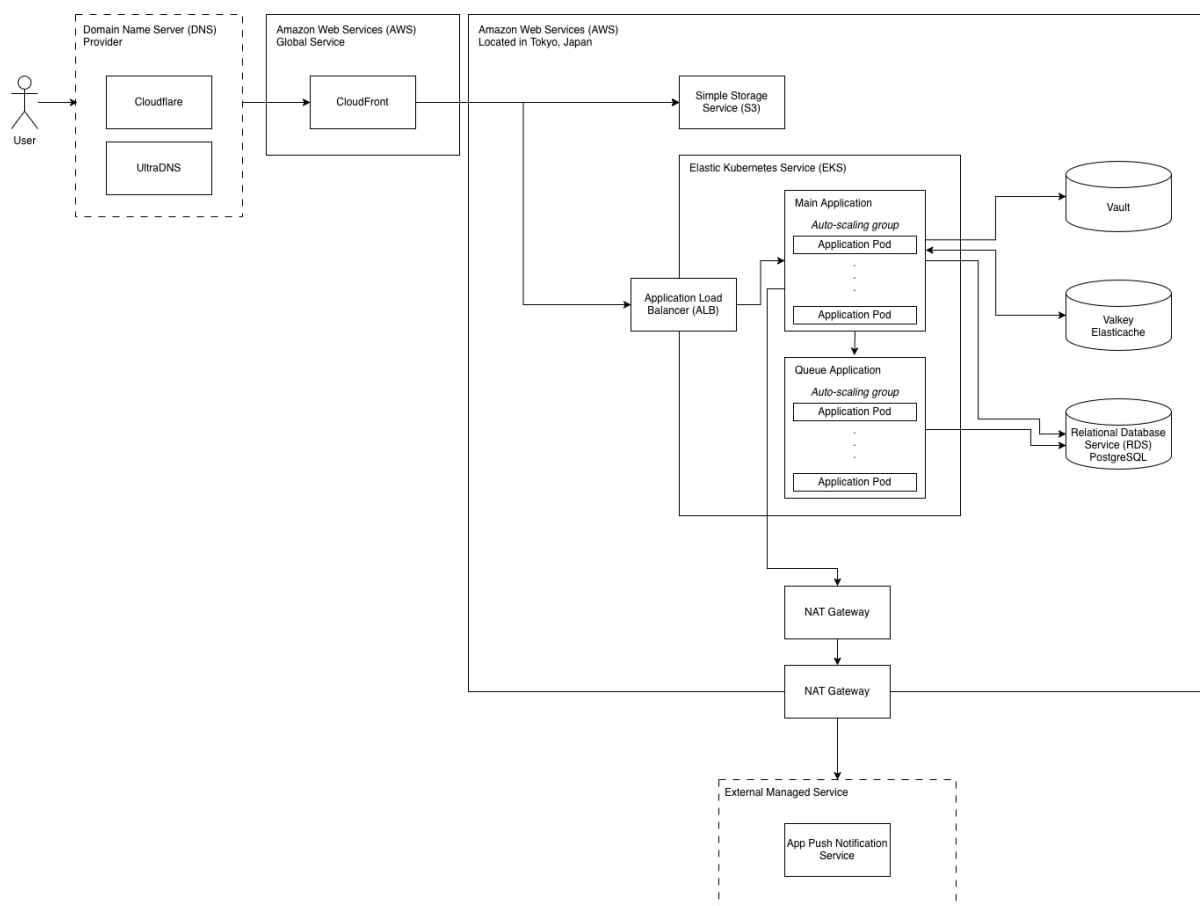
NodeJS is able to quickly spin up simple services that serve a reasonable amount of load. Its single threaded-ness is simple to manage and can properly scale if written properly. By leveraging on websockets and familiar databases, the team will be able to deliver the project

on time. However, its main drawback is its comparatively weaker throughput estimated at around 1000 - 5000 users.

Solution 2b - NodeJS + Valkey Streams + Database Queue

In order to increase the upper ceiling of NodeJS, external in memory stores like Valkey, and a microservice architecture can be adopted. Valkey possesses powerful features like Pub/Sub and Valkey Streams that is able to take the heavy lifting away from NodeJS, allowing the application to scale much faster.

On top of that, in order to alleviate database load, a simple queue like (BullMQ) [\[2\]](#) can be built to batch or throttle messages into groups. The detailed architecture diagram is depicted as such:



By setting up elaborate microservices and migrating core processing onto an external cache like Valkey, it is estimated that the throughput could potentially reach 100,000 active users.

Others

Several alternative solutions were also explored when designing the software solution.

Bun Runtime

Bun [3] is a drop-in replacement and all-in-one tool for Node JS, rewritten in zig. It took the Javascript community by storm after boasting tremendous performance improvements. (~3x NodeJS as server, ~5x ws library as websocket, ~2x esbuild).

Bun was initially a heavy contender during the architecture consideration as its websocket is built on uWebSockets [4], a low-level websocket implementation written entirely in C and C++. Although uWebSockets can also be implemented in NodeJS directly, it is much harder to implement and requires large amounts of configuration to get going.

However, Bun was finally dropped due to the fact that many important NodeJS libraries and functions might not work properly in Bun. Based on experience from other projects and online recounts, libraries like image processing often break within Bun.

Tencent RTC

Tencent RTC [5] is a managed service that powers wechat and billions of its users. They provide easy to implement SDK and ready to go frontend applications. Adopting this would also allow quicker approval from China's App Approval Team. However, unique features raised specific to this project prevents the team from proceeding.

The deal breaker for dropping Tencent RTC is that it requires a direct connection from Client to Tencent's servers. This will result in users being able to access unauthorized features like user to user direct messaging.

Recommendation

Recommended Solution - Solution 2b

As the expected load given by the Project Manager ranges in the thousands, a NodeJS Solution will be more than sufficient. This also allows cost to be much lower as compared to adopting the full suite of services required of an open sourced chat framework. NodeJS, Valkey and Postgres are also familiar technologies, allowing the team to be able to serve technical requests or debugging.

Although it would be much simpler to build Solution 2a, the maximum throughput that it can support is too close for comfort. By migrating the core of chat streaming into something like Valkey and building a simple queue system, we are able to easily expand our system's capabilities by a certain order of magnitude.

References

- [1] openimsdk. (n.d.). *open-im-server* [Computer software]. GitHub. Retrieved January 19, 2026, from <https://github.com/openimsdk/open-im-server>
- [2] BullMQ. (n.d.). *BullMQ: Background jobs processing and message queue for Node.js*. Retrieved January 19, 2026, from <https://bullmq.io/>

- [3] Bun. (n.d.). *Bun — A fast all-in-one JavaScript runtime*. Retrieved January 20, 2026, from <https://bun.com/>
- [4] uNetworking. (n.d.). *uWebSockets* [Computer software]. Retrieved January 20, 2026, from <https://github.com/uNetworking/uWebSockets>
- [5] Tencent Real-Time Communication. (n.d.). *TRTC — Real-time audio and video communication platform*. Retrieved January 20, 2026, from <https://trtc.io/>